

Dyad Backend API Documentation

Version: v1

Base path: /api

Generated at: 2026-06-29T08:54:00.831Z

Notes

- All request and response examples reflect the current Express route handlers in this repository.
- Most endpoints return JSON. The verify-email endpoint returns plain text, and the documentation endpoint returns a downloadable PDF.
- Fields listed as aliases are accepted interchangeably by the backend, even if only one naming style is recommended for new clients.

Authentication and Users

Registration, login, OTP verification, password reset, and user profile endpoints.

POST /api/register

Create a new user account.

Authentication: Public

Registers a normal user. `email`, `password`, `npi`, and `phone` are required.

Example request body

```
{
  "email": "premkumar.jaguar@example.com",
  "password": "StrongPassword123",
  "firstName": "Premkumar",
  "lastName": "Vellaisamy",
  "npi": "1234567890",
  "phone": "9715789136",
  "emailVerified": false
}
```

Response 200 - User created successfully.

Example body

```
{
  "success": true,
  "message": "User registered successfully. Please verify your email."
}
```

Response 409 - Email or NPI already exists.

Example body

```
{
  "success": false,
  "message": "Email already exists"
}
```

POST /api/login

Authenticate a user and issue JWT tokens.

Authentication: Public

Example request body

```
{
  "email": "premkumar.jaguar@example.com",
  "password": "StrongPassword123"
}
```

Response 200 - Login successful.

Example body

```
{
  "success": true,
  "accessToken": "<jwt-access-token>",
  "refreshToken": "<jwt-refresh-token>",
  "user": {
    "id": 12,
    "email": "premkumar.jaguar@example.com",
    "role": "user"
  }
}
```

Response 403 - Email is not verified.

Example body

```
{
  "success": false,
  "message": "Please verify your email first before logging in"
}
```

POST /api/refresh

Exchange a refresh token for a fresh access token.

Authentication: Public

Example request body

```
{
  "refreshToken": "<jwt-refresh-token>"
}
```

Response 200 - Refresh successful.

Example body

```
{
  "success": true,
  "accessToken": "<new-jwt-access-token>"
}
```

Response 403 - Refresh token is invalid or expired.

Example body

```
{
  "success": false,
  "message": "Invalid refresh token"
}
```

GET /api/profile

Fetch the currently authenticated user profile.

Authentication: Bearer token required

Headers

- **Authorization (required):** JWT access token returned by the login endpoint.

Response 200 - Profile returned.

Example body

```
{
  "success": true,
  "data": {
    "id": 12,
    "first_name": "Premkumar",
    "last_name": "Vellaisamy",
    "email": "premkumar.jaguar@example.com",
    "npi": "1234567890",
    "phone": "9715789136",
    "role": "user"
  }
}
```

Response 401 - Missing or invalid bearer token.

Example body

```
{
  "success": false,
  "message": "No token provided"
}
```

GET /api/verify-email

Verify a user's email using a token from the email link.

Authentication: Public

Query parameters

- **token (required):** Email verification token.

Response 200 - Email verified successfully.

Example body

```
Email verified successfully
```

Response 400 - Token missing or invalid.

Example body

```
Invalid or expired verification token
```

POST /api/send-email-otp

Send a registration OTP to the provided email address.

Authentication: Public

Example request body

```
{
  "email": "premkumar.jaguar@example.com"
}
```

Response 200 - OTP email sent.

Example body

```
{
  "success": true,
  "message": "OTP sent to your email address"
}
```

POST /api/verify-otp

Validate the registration OTP and mark the email as verified.

Authentication: Public

Example request body

```
{
  "email": "premkumar.jaguar@example.com",
  "otp": "123456"
}
```

Response 200 - OTP accepted.

Example body

```
{
  "success": true,
  "message": "OTP verified successfully",
  "email_verified": true,
  "user": {
    "id": 12,
    "email_verified": true
  }
}
```

Response 400 - OTP missing, invalid, or expired.

Example body

```
{
  "success": false,
  "message": "Invalid OTP"
}
```

POST /api/send-reset-otp

Send a password-reset OTP to the email address.

Authentication: Public

Example request body

```
{
  "email": "premkumar.jaguar@example.com"
}
```

Response 200 - Reset OTP flow completed.

Example body

```
{
  "success": true,
  "message": "Password reset OTP sent to your email address"
}
```

POST /api/reset-password

Reset a user's password after OTP verification.

Authentication: Public

Example request body

```
{
  "email": "premkumar.jaguar@example.com",
  "otp": "123456",
  "newPassword": "EvenStrongerPassword123"
}
```

Response 200 - Password changed.

Example body

```
{
  "success": true,
  "message": "Password reset successfully. You can now login with your new password."
}
```

Response 400 - OTP or password validation failed.

Example body

```
{
  "success": false,
  "message": "Invalid OTP"
}
```

POST /api/check-email

Check whether a user email already exists.

Authentication: Public

Example request body

```
{
  "email": "premkumar.jaguar@example.com"
}
```

Response 200 - Existence check completed.

Example body

```
{
  "success": true,
  "exists": true,
  "message": "Email exists"
}
```

GET /api/users

List all users.

Authentication: Bearer token required

Headers

- **Authorization (required):** JWT access token returned by the login endpoint.

Response 200 - User list returned.

Example body

```
{
  "success": true,
  "data": [
    {
      "id": 12,
      "first_name": "Premkumar",
      "last_name": "Vellaisamy",
      "email": "premkumar.jaguar@example.com",
      "npi": "1234567890",
      "phone": "9715789136",
      "role": "user",
      "email_verified": true,
      "created_at": "2026-06-29T06:30:00.000Z"
    }
  ],
  "count": 1
}
```

GET /api/userslist

Return the public list of users without authentication.

Authentication: Public

Response 200 - User list returned.

Example body

```
{
  "success": true,
  "data": [
    {
      "id": 12,
      "first_name": "Premkumar",
      "last_name": "Vellaisamy",
      "email": "premkumar.jaguar@example.com"
    }
  ],
  "count": 1
}
```

PUT /api/users/:id

Update selected user fields.

Authentication: Bearer token required

Headers

- **Authorization (required):** JWT access token returned by the login endpoint.

Path parameters

- **id (required):** Numeric user id.

Example request body

```
{
  "first_name": "Prem",
  "last_name": "V",
  "email": "premkumar.updated@example.com",
  "npi": "1234567890",
  "phone": "9715789136",
  "role": "admin",
  "email_verified": true
}
```

Response 200 - User updated.

Example body

```
{
  "success": true,
  "message": "User updated successfully",
  "data": {
    "id": 12,
    "email": "premkumar.updated@example.com",
    "role": "admin"
  }
}
```

Response 404 - User id not found.

Example body

```
{
  "success": false,
  "message": "User not found"
}
```

POST /api/admin/login

Authenticate an admin account and issue admin JWT tokens.

Authentication: Public

Example request body

```
{
  "username": "superadmin",
  "password": "AdminPassword123"
}
```

Response 200 - Admin login successful.

Example body

```
{
  "success": true,
  "message": "Admin login successful",
  "accessToken": "<admin-access-token>",
  "refreshToken": "<admin-refresh-token>",
  "admin": {
    "id": 1,
    "username": "superadmin",
    "role": "admin"
  }
}
```

Contact, Lead Capture, and Email

Lead capture, invitation, and transactional email endpoints.

POST /api/contact-requests

Create a new contact request.

Authentication: Public

Example request body

```
{
  "name": "John Doe",
  "phoneNumber": "9715789136",
  "email": "john.doe@example.com",
  "organization": "Healthcare Facility",
  "message": "I would like to learn more about your services.",
  "scheduledTime": "2026-07-05T10:00:00.000Z"
}
```

Response 201 - Contact request saved.

Example body

```
{
  "success": true,
  "message": "Contact request submitted successfully",
  "data": {
    "id": 101,
    "name": "John Doe",
    "phone_number": "9715789136",
    "email": "john.doe@example.com",
    "organization": "Healthcare Facility",
    "message": "I would like to learn more about your services.",
    "scheduled_time": "2026-07-05T10:00:00.000Z",
    "status": "pending"
  }
}
```

GET /api/contact-requests

List contact requests with optional pagination and filters.

Authentication: Public

Query parameters

- **page:** Defaults to 1.
- **limit:** Defaults to 10.
- **status:** Filter by workflow status.
- **organization:** Filter by organization name.
- **search:** Text search across key fields.

Response 200 - Paginated contact requests returned.

Example body

```
{
  "success": true,
  "data": [
    {
      "id": 101,
      "name": "John Doe",
      "email": "john.doe@example.com",
      "status": "pending"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "pages": 1
  }
}
```

GET /api/contact-requests/:id

Fetch a single contact request by id.

Authentication: Public

Path parameters

- **id (required):** Contact request id.

Response 200 - Contact request returned.

Example body

```
{
  "success": true,
  "data": {
    "id": 101,
    "name": "John Doe",
    "email": "john.doe@example.com",
    "status": "pending"
  }
}
```

Response 404 - Contact request not found.

Example body

```
{
  "success": false,
  "message": "Contact request not found"
}
```

PATCH /api/contact-requests/:id/status

Update the workflow status of a contact request.

Authentication: Public

Path parameters

- **id (required):** Contact request id.

Example request body

```
{
  "status": "contacted"
}
```

Response 200 - Status updated.

Example body

```
{
  "success": true,
  "message": "Contact request status updated successfully"
}
```

DELETE /api/contact-requests/:id

Delete a contact request.

Authentication: Public

Path parameters

- **id (required):** Contact request id.

Response 200 - Contact request deleted.

Example body

```
{
  "success": true,
  "message": "Contact request deleted successfully"
}
```


POST /api/send-onboarding-schedule-confirmation

Send a schedule confirmation email with meeting details.

Authentication: Public

Example request body

```
{
  "to": "john.doe@example.com",
  "subject": "You have scheduled Meeting with Dyad Practice Solutions",
  "contactName": "John Doe",
  "dateTime": "2026-07-05T10:00:00.000Z",
  "meetingLink": "https://meet.google.com/abc-defg-hij",
  "calendarEmail": "dyadcontactrequest@gmail.com",
  "joinMeetingLabel": "Join Google Meet"
}
```

Response 200 - Email sent.

Example body

```
{
  "success": true,
  "message": "Onboarding schedule confirmation sent successfully",
  "messageId": "<smtp-message-id>",
  "to": "john.doe@example.com"
}
```

POST /api/send-email/sendgrid

Send an email and mark the matching early-access row as emailed.

Authentication: Public

Example request body

```
{
  "to": "john.doe@example.com",
  "subject": "Welcome to Dyad",
  "html": "<p>Your invite is ready.</p>",
  "text": "Your invite is ready."
}
```

Response 200 - Email sent.

Example body

```
{
  "success": true,
  "message": "Email sent successfully",
  "messageId": "<smtp-message-id>"
}
```

Scheduling and Calendar

Google Calendar and Calendly-based scheduling endpoints.

POST /api/create-event

Create a Google Calendar event and optionally attach a Google Meet link.

Authentication: Public

Example request body

```
{
  "title": "Intro Call",
  "dateTime": "2026-07-05T10:00:00.000Z",
  "onboardingId": "onb_123456",
  "description": "Requested from onboarding flow",
  "createMeetLink": true
}
```

Response 201 - Event created successfully.

Example body

```
{
  "success": true,
  "message": "Event created!",
  "eventId": "1m7q8exampleeventid",
  "callEventId": "1m7q8exampleeventid",
  "meetingLink": "https://meet.google.com/abc-defg-hij",
  "meetingId": "abc-defg-hij",
  "meetingLinkSource": "google_calendar",
  "eventLink": "https://calendar.google.com/calendar/event?eid=...",
  "start": {
    "dateTime": "2026-07-05T10:00:00.000Z",
    "timeZone": "Asia/Kolkata"
  },
  "end": {
    "dateTime": "2026-07-05T10:30:00.000Z",
    "timeZone": "Asia/Kolkata"
  }
}
```

PATCH | PUT | POST /api/update-event

Update or reschedule an existing calendar event. The same handler accepts three verbs.

Authentication: Public

Recommended fields are shown below; several aliases are also accepted by the backend.

Example request body

```
{
  "eventId": "1m7q8exampleeventid",
  "title": "Rescheduled Intro Call",
  "dateTime": "2026-07-05T11:00:00.000Z",
  "description": "Updated description",
  "createMeetLink": true,
  "onboardingId": "onb_123456"
}
```

Response 200 - Event updated successfully.

Example body

```
{
  "success": true,
  "message": "Event updated successfully",
  "eventId": "1m7q8exampleeventid",
  "meetingLink": "https://meet.google.com/abc-defg-hij",
  "meetingId": "abc-defg-hij"
}
```

Response 404 - Calendar event not found.

Example body

```
{
  "success": false,
  "message": "Event not found"
}
```

GET /api/calendar-events

Return event start and end times for a single date.

Authentication: Public

Query parameters

- **date (required):** Date in YYYY-MM-DD format.

Response 200 - Events returned.

Example body

```
{
  "events": [
    {
      "start": "2026-07-05T10:00:00.000Z",
      "end": "2026-07-05T10:30:00.000Z"
    }
  ]
}
```

GET /api/calendar/available-slots

Compute available Google Calendar slots for a day.

Authentication: Public

Query parameters

- **date (required):** Date in YYYY-MM-DD format.

Response 200 - Available slots returned.

Example body

```
{
  "success": true,
  "date": "2026-07-05",
  "calendarId": "dyadcontactrequest@gmail.com",
  "timeZone": "Asia/Kolkata",
  "slotDurationMinutes": 30,
  "availableSlots": [
    {
      "start": "2026-07-05T09:00:00+05:30",
      "end": "2026-07-05T09:30:00+05:30"
    }
  ]
}
```

GET /api/slots

Fetch Calendly availability between a start and end time.

Authentication: Public

Query parameters

- **start (required):** ISO start date-time.

- **end (required):** ISO end date-time.

Response 200 - Calendly slots returned.

Example body

```
{
  "success": true,
  "data": [
    {
      "start_time": "2026-07-05T10:00:00.000Z",
      "scheduling_url": "https://calendly.com/..."
    }
  ]
}
```

POST /api/book

Create a Calendly booking.

Authentication: Public

Example request body

```
{
  "name": "John Doe",
  "email": "john.doe@example.com",
  "start_time": "2026-07-05T10:00:00.000Z"
}
```

Response 200 - Booking created.

Example body

```
{
  "success": true,
  "message": "Booking created successfully",
  "data": {
    "uri": "https://api.calendly.com/scheduled_events/..."
  }
}
```

GET /api/event-types

Return Calendly event types configured for the token owner.

Authentication: Public

Response 200 - Event types returned.

Example body

```
{
  "success": true,
  "data": [
    {
      "name": "Discovery Call",
      "uri": "https://api.calendly.com/event_types/..."
    }
  ]
}
```

POST /api/calendly/webhook

Receive Calendly webhook events. Primarily handles invitee-created events.

Authentication: Public

Example request body

```
{
  "event": "invitee.created",
  "payload": {
    "email": "john.doe@example.com",
    "name": "John Doe"
  }
}
```

Response 200 - Webhook accepted.

Example body

```
{}
```

POST /api/calls-scheduled-admin

Persist an admin-managed scheduled call record.

Authentication: Public

Aliases like `contact\_name`, `event\_title`, and `meeting\_id` are also accepted.

Example request body

```
{
  "email": "john.doe@example.com",
  "contactName": "John Doe",
  "eventTitle": "Admin Follow-up Call",
  "emailSubject": "Your Dyad call is scheduled",
  "callType": "onboarding",
  "mailDescription": "Please join on time.",
  "dateTime": "2026-07-05T10:00:00.000Z",
  "meetingId": "abc-defg-hij"
}
```

Response 201 - Scheduled call saved.

Example body

```
{
  "success": true,
  "message": "Scheduled call saved successfully",
  "data": {
    "id": 14,
    "email": "john.doe@example.com",
    "contactName": "John Doe",
    "eventTitle": "Admin Follow-up Call",
    "emailSubject": "Your Dyad call is scheduled",
    "callType": "onboarding",
    "mailDescription": "Please join on time.",
    "dateTime": "2026-07-05T10:00:00.000Z",
    "meetingId": "abc-defg-hij",
    "meetingLink": "https://meet.google.com/abc-defg-hij"
  }
}
```

GET /api/calls-scheduled-admin

List admin scheduled calls with filters.

Authentication: Public

Query parameters

- **email:** Filter by email.
- **meetingId:** Filter by meeting id.
- **callType:** Filter by call type.
- **limit:** Defaults to 100, max 500.
- **offset:** Defaults to 0.

Response 200 - Scheduled calls returned.

Example body

```
{
  "success": true,
  "total": 1,
  "data": [
    {
      "id": 14,
      "email": "john.doe@example.com",
      "meetingId": "abc-defg-hij",
      "meetingLink": "https://meet.google.com/abc-defg-hij"
    }
  ]
}
```

Early Access and Onboarding

Lead qualification, onboarding data capture, and readback endpoints.

POST /api/api-early-access

Submit a new early-access request.

Authentication: Public

Example request body

```
{
  "practiceName": "ABC Clinic",
  "contactName": "Dr. Jane Smith",
  "phoneNumber": "9715789136",
  "email": "jane.smith@example.com",
  "title": "Owner",
  "practiceType": "Behavioral Health",
  "npi": "1234567890",
  "providers": 5,
  "locations": 2,
  "claimVolume": "500-1000"
}
```

Response 201 - Early-access request created.

Example body

```
{
  "success": true,
  "message": "Early access request submitted successfully",
  "data": {
    "id": 21,
    "email": "jane.smith@example.com",
    "status": "pending"
  }
}
```

POST /api/api-early-access/check-email

Check whether an early-access email already exists.

Authentication: Public

Example request body

```
{
  "email": "jane.smith@example.com"
}
```

Response 200 - Existence check returned.

Example body

```
{
  "success": true,
  "exists": true,
  "message": "Email already registered for early access",
  "status": "pending",
  "submittedAt": "2026-06-29T08:00:00.000Z"
}
```

POST /api/api-early-access/check-npi

Check whether an NPI already exists in early access.

Authentication: Public

Example request body

```
{
  "npi": "1234567890"
}
```

Response 200 - Existence check returned.

Example body

```
{
  "success": true,
  "exists": false,
  "message": "NPI is available"
}
```

PATCH /api/api-early-access/:id

Update early-access status and invitation flags.

Authentication: Public

Path parameters

- **id (required):** Early-access request id.

Example request body

```
{
  "betaInvite": true,
  "invitationSent": true,
  "status": "approved"
}
```

Response 200 - Early-access record updated.

Example body

```
{
  "success": true,
  "message": "Early access request updated successfully"
}
```

GET /api/api-early-access

List all early-access requests.

Authentication: Public

Response 200 - Early-access requests returned.

Example body

```
{
  "success": true,
  "total": 1,
  "data": [
    {
      "id": 21,
      "email": "jane.smith@example.com",
      "status": "pending"
    }
  ]
}
```

POST /api/api-early-access/send-invite

Send invite emails to multiple early-access requests by id.

Authentication: Public

Example request body

```
{
  "ids": [
    21,
    22
  ],
  "subject": "Your Dyad invite",
  "html": "<p>Welcome to Dyad.</p>",
  "text": "Welcome to Dyad."
}
```

Response 200 - Invite emails sent.

Example body

```
{
  "success": true,
  "message": "Invitation emails sent successfully",
  "sent": 2
}
```

POST /api/api-early-access/send-invite-email

Send one custom invite email.

Authentication: Public

Example request body

```
{
  "email": "jane.smith@example.com",
  "subject": "You are invited",
  "html": "<p>Join the beta.</p>",
  "text": "Join the beta."
}
```

Response 200 - Custom invite email sent.

Example body

```
{
  "success": true,
  "message": "Invite email sent successfully",
  "messageId": "<smtp-message-id>"
}
```

POST /api/onboarding/check-email

Check whether an onboarding record already exists for an email address.

Authentication: Public

Example request body

```
{
  "email": "jane.smith@example.com"
}
```

Response 200 - Existence check returned.

Example body

```
{
  "success": true,
  "exists": true,
  "onboardingId": "onb_123456",
  "status": "Onboarding",
  "message": "Onboarding record already exists"
}
```


POST /api/onboarding-client/check-npi

Check whether an onboarding record already exists for an NPI.

Authentication: Public

Example request body

```
{
  "npi": "1234567890"
}
```

Response 200 - Existence check returned.

Example body

```
{
  "success": true,
  "exists": false,
  "message": "NPI is available"
}
```

POST /api/onboarding/step/:step

Save step payloads for onboarding steps 1 through 6.

Authentication: Public

The backend stores arbitrary JSON payloads. For steps 2-6, provide `onboardingId` unless the record can be inferred from identifiers.

Path parameters

- **step (required):** Step number from 1 to 6.

Example request body

```
{
  "onboardingId": "onb_123456",
  "contactEmail": "jane.smith@example.com",
  "contactName": "Dr. Jane Smith",
  "npi": "1234567890",
  "meetingId": "abc-defg-hij",
  "stepPayload": {
    "plan": "monthly",
    "amount": 99
  }
}
```

Response 200 - Step saved successfully.

Example body

```
{
  "success": true,
  "message": "Onboarding step saved successfully",
  "onboardingId": "onb_123456",
  "status": "Onboarding",
  "meetingId": "abc-defg-hij",
  "meetingLink": "https://meet.google.com/abc-defg-hij",
  "completedSteps": [
    1,
    2,
    3
  ],
  "data": {
    "onboardingId": "onb_123456"
  }
}
```

GET /api/onboarding

List all onboarding records.

Authentication: Public

Response 200 - Onboarding records returned.

Example body

```
{
  "success": true,
  "total": 1,
  "data": [
    {
      "onboardingId": "onb_123456",
      "status": "Onboarding",
      "completedSteps": [
        1,
        2,
        3
      ]
    }
  ]
}
```

GET /api/onboarding/:onboardingId

Fetch a single onboarding record with aggregated step data.

Authentication: Public

Path parameters

- **onboardingId (required):** Application onboarding id.

Response 200 - Onboarding record returned.

Example body

```
{
  "success": true,
  "onboardingId": "onb_123456",
  "meetingId": "abc-defg-hij",
  "meetingLink": "https://meet.google.com/abc-defg-hij",
  "completedSteps": [
    1,
    2,
    3
  ],
  "data": {
    "step_1_payload": {},
    "step_2_payload": {}
  }
}
```

Response 404 - Onboarding record not found.

Example body

```
{
  "success": false,
  "message": "Onboarding record not found"
}
```

GET /api/onboarding/:onboardingId/step/:step

Fetch a specific onboarding step payload.

Authentication: Public

Path parameters

- **onboardingId (required):** Application onboarding id.
- **step (required):** Step number from 1 to 6.

Response 200 - Step payload returned.

Example body

```
{
  "success": true,
  "onboardingId": "onb_123456",
  "step": 3,
  "meetingId": "abc-defg-hij",
  "meetingLink": "https://meet.google.com/abc-defg-hij",
  "data": {
    "amount": 99,
    "plan": "monthly"
  }
}
```

Utility Data and Validation

reCAPTCHA verification and external registry lookups.

POST /api/verify-recaptcha

Verify a reCAPTCHA token against the configured secret.

Authentication: Public

Example request body

```
{
  "token": "<recaptcha-token>"
}
```

Response 200 - Verification completed.

Example body

```
{
  "success": true,
  "message": "reCAPTCHA verified successfully",
  "errorCodes": []
}
```

POST /api/npi/registry

Look up a provider from the CMS NPI registry.

Authentication: Public

Example request body

```
{
  "npi": "1234567890"
}
```

Response 200 - Provider data returned.

Example body

```
{
  "success": true,
  "message": "NPI record found",
  "data": {
    "npi": "1234567890",
    "enumeration_type": "NPI-1",
    "basic": {
      "first_name": "Jane",
      "last_name": "Smith"
    },
    "addresses": [],
    "taxonomies": [],
    "identifiers": [],
    "other_names": []
  }
}
```

```
}
```

Response 404 - No record found for the NPI.

Example body

```
{
  "success": false,
  "message": "No NPI record found"
}
```

GET /api/taxonomies

Return the provider taxonomy code list.

Authentication: Public

Response 200 - Taxonomy list returned.

Example body

```
{
  "success": true,
  "total": 2,
  "data": [
    "261QH0100X",
    "208D00000X"
  ]
}
```

Zoho Payments and Subscriptions

Zoho customer creation, payment session creation, payment verification, and recurring billing.

POST /api/customer

Create or reuse a Zoho customer for a local owner record.

Authentication: Public

Example request body

```
{
  "onboardingId": "onb_123456",
  "name": "Dr. Jane Smith",
  "email": "jane.smith@example.com",
  "phone": "9715789136"
}
```

Response 200 - Customer mapping returned.

Example body

```
{
  "customerId": "903000000012345",
  "isNew": true
}
```

POST /api/create-session

Create a Zoho ACH payment session.

Authentication: Public

Example request body

```
{
  "customerId": "903000000012345",
  "amount": 99,
  "currency": "USD",
  "plan": "monthly"
}
```

Response 200 - Payment session returned for Zoho widget initialization.

Example body

```
{
  "session_id": "903000000054321",
  "payments_session_id": "903000000054321",
  "customer_id": "903000000012345",
  "account_id": "926398629",
  "api_key": "<zoho-api-key>",
  "plan": "monthly",
  "amount": 99,
  "currency": "USD"
}
```

POST /api/verify-payment

Verify the status of a Zoho payment after widget completion.

Authentication: Public

Example request body

```
{
  "payment_id": "903000000078901"
}
```

Response 200 - Payment status returned.

Example body

```
{
  "success": true,
  "payment_id": "903000000078901",
  "status": "succeeded",
  "customer_id": "903000000012345",
  "payment_method_id": "903000000011111",
  "amount": 99,
  "currency": "USD",
  "data": {
    "payment_id": "903000000078901"
  }
}
```

POST /api/create-payment-method-session

Create a Zoho session for saving an ACH payment method.

Authentication: Public

Example request body

```
{
  "customerId": "903000000012345",
  "description": "Save ACH for recurring subscription"
}
```

Response 200 - Payment method session returned.

Example body

```
{
  "payment_method_session_id": "903000000066666",
  "customer_id": "903000000012345",
  "account_id": "926398629",
  "api_key": "<zoho-api-key>",
  "widget": {
    "payment_method": "ach_debit",
    "transaction_type": "add",
    "customer_id": "903000000012345",
    "payment_method_session_id": "903000000066666"
  }
}
```

POST /api/save-subscription

Persist the recurring subscription after a successful payment and payment-method capture.

Authentication: Public

Example request body

```
{
  "onboardingId": "onb_123456",
  "customerId": "903000000012345",
  "payment_id": "903000000078901",
  "payment_method_id": "903000000011111",
  "plan": "monthly",
  "amount": 99,
  "currency": "USD"
}
```

Response 201 - Subscription created in the local database.

Example body

```
{
  "success": true,
  "isNew": true,
  "data": {
    "owner_id": "onb_123456",
    "zoho_customer_id": "903000000012345",
    "zoho_payment_id": "903000000078901",
    "zoho_payment_method_id": "903000000011111",
    "plan": "monthly",
    "amount": "99.00",
    "currency": "USD",
    "status": "active"
  }
}
```

GET /api/subscription/:ownerId

Fetch the saved Zoho subscription for a user or onboarding owner id.

Authentication: Public

Path parameters

- **ownerId (required):** Local userId or onboardingId.

Response 200 - Subscription returned.

Example body

```
{
  "success": true,
  "data": {
    "owner_id": "onb_123456",
    "status": "active",
    "next_charge": "2026-08-05T10:00:00.000Z"
  }
}
```

Response 404 - No subscription exists for the owner.

Example body

```
{
  "error": "No subscription found"
}
```

POST /api/run-recurring-billing

Trigger charging for all due subscriptions.

Authentication: Public, optionally guarded by cron secret

Headers

- **x-cron-secret:** Required only when `CRON\_SECRET` or `ZOHO\_CRON\_SECRET` is configured.

Example request body

```
{
  "cronSecret": "<cron-secret>"
}
```

Response 200 - Recurring billing batch completed.

Example body

```
{
  "success": true,
  "processed": 1,
  "results": [
    {
      "subscriptionId": 1,
      "success": true,
      "paymentId": "903000000099999",
      "nextCharge": "2026-09-05T10:00:00.000Z"
    }
  ]
}
```

Response 401 - Secret is missing or invalid when protection is enabled.

Example body

```
{
  "error": "Unauthorized"
}
```


